

Trabajo Práctico de Programación B
Programación en Lenguaje C

Ejercicios

1. Considerar los siguientes planteos en el contexto del lenguaje C:
 - a) ¿Cómo se definen las variables de tipo puntero?
 - b) ¿Qué estructuras de datos pueden ser apuntadas por una variable de tipo puntero?
 - c) ¿Es posible declarar una variable de tipo puntero sin especificar a qué estructura de datos referenciará?
 - d) ¿La constante NULL, de qué tipo es?
 - e) ¿Por qué los punteros permiten simular un pasaje de parámetros por referencia?
¿Qué operadores son necesarios para tal fin?
2. Teniendo en cuenta los siguientes fragmentos de código en C, realice una traza que le permita responder cuál es el valor de cada variable, al momento de llegar a cada checkpoint.

```
void main(){
    int* x;
    int y, z = 3;
    x = &z;
    y = *x;
    //checkpoint 1
    y++;
    //checkpoint 2
    (*x) ++;
    //checkpoint 3
}
```

```
void main(){
    int* x;
    int y;
    x = &y;
    *x = 3;
    //checkpoint 1
    y++;
    //checkpoint 2
    (*x) ++;
}
```

3. Teniendo en cuenta los siguientes fragmentos de código en C, realice una traza que, al momento de llegar a cada checkpoint, le permita responder:
 - ¿Para cuáles de las variables definidas es modificable su valor?
 - ¿Qué valor tiene cada una?

a)

```
void main(){
    double myVar = 123.4;
    f(myVar);
    //checkpoint3
}
```

```
void f(double formal){
    //checkpoint1
    formal = formal + 56.75;
    //checkpoint2
}
```

b)	<pre>void main(){ double myVar = 123.4; f(&myVar); //checkpoint3 }</pre>	<pre>void f(double* formal){ //checkpoint1 *formal = *formal + 56.75; //checkpoint2 }</pre>
c)	<pre>void main(){ double myVar = 123.4; double* p; p = &myVar; f(*p); //checkpoint3 }</pre>	<pre>void f(double formal){ //checkpoint1 formal = formal + 56.75; //checkpoint2 }</pre>
d)	<pre>void main(){ double myVar = 123.4; double* p; p = &myVar; f(p); //checkpoint3 }</pre>	<pre>void f(double* formal){ //checkpoint1 *formal = *formal + 56.75; //checkpoint2 }</pre>
e)	<pre>void main(){ double myVar = 123.4; double* p; p = &myVar; f(&p); //checkpoint4 }</pre>	<pre>void f(double** formal){ //checkpoint1 **formal = **formal + 56.75; *formal = NULL; //checkpoint2 formal = NULL; //checkpoint3 }</pre>

4. Supongamos un programa que, haciendo uso de la función `tri`, triplica el valor de un entero. De las soluciones planteadas a continuación todas compilan, sin embargo, no todas las soluciones son correctas (aunque algunas funcionen). Analice cada una de ellas e identifique cuál es el problema.

a)	<pre>void main(){ int a = 34; int a3; int* p; p = tri(a); a3 = *p; }</pre>	<pre>int* tri(int x){ int res; res = 3 * x; return &res; }</pre>
----	--	--

b)	<pre>void main(){ int a = 34; int a3; int* p; p = tri(a); a3 = *p; }</pre>	<pre>int* tri(int x){ int* res; *res = 3 * x; return res; }</pre>
c)	<pre>void main(){ int a = 34; int a3; a3 = tri(a); }</pre>	<pre>int tri(int x){ int res = 3 * x; return res; }</pre>
d)	<pre>void main(){ int a = 34; tri(&a); }</pre>	<pre>void tri(int* x){ int res; res = *x * 3; *x = res; }</pre>
e)	<pre>void main(){ int* a; *a = 34; tri(a); }</pre>	<pre>void tri(int* x){ int res; res = *x * 3; *x = res; }</pre>
f)	<pre>void main(){ int* a; int b; a = &b; *a = 34; tri(a); }</pre>	<pre>void tri(int* x){ int res; res = *x * 3; *x = res; }</pre>

5. Diseñar funciones o procedimientos para implementar las siguientes operaciones sobre arreglos de números enteros:

- Un procedimiento que inicialice al azar los primeros n elementos de un arreglo recibido como argumento.
- Un procedimiento que ordene en forma ascendente los primeros n elementos de un arreglo recibido como argumento.
- Una función que determine si los primeros n elementos de un arreglo recibido como argumento se encuentran ordenados de forma ascendente.
- Un procedimiento que reciba dos arreglos de enteros A_1 y A_2 , cada uno con una cierta cantidad de elementos previamente ordenados de forma ascendente, y que retorne un nuevo arreglo resultante de intercalar de forma ordenada y sin repetición los elementos de A_1 y A_2 .

6. Implemente un procedimiento que, dado un arreglo de enteros *arr* y dos enteros *a* y *n*, modifique el arreglo *arr* de forma tal que cada aparición del entero *a* sea reemplazada por el entero *n*. Analice dos variantes, una en función de cada uno de los siguientes prototipos:

- `void reemplazar(int arr [], int a, int n);`
- `void reemplazar(int * arr, int a, int n, int size);`

Compruebe el correcto funcionamiento de las dos variantes considerando reemplazar el entero 3 por el entero 0 en un arreglo inicializado con los valores 1,2,3,4,5.

7. Dado el siguiente fragmento de código:

```
#include <stdio.h>
#include <stdlib.h>

void invertir_arr_num(int arr [], int size){
    int f = 0, c = size-1;
    int aux;
    while(f < c){
        aux = arr[f];
        arr[f++] = arr[c];
        arr[c--] = aux;
    }
}

void invertir_arr_punt(int ** arr, int size){
    int ** cola = arr + size - 1;
    int * aux;
    while(arr != cola){
        aux = *arr;
        *arr = *cola;
        *cola = aux;
        if ((++arr)!=cola)
            cola--;
    }
}

int main(){
    int longitud = 3;
    int arr_num [longitud];
    int * arr_punt [longitud];

    // INICIALIZACIÓN
    int i = 0;
    for(i=0; i<longitud; i++){
        arr_num[i] = i+1;
        arr_punt[i] = &arr_num[i];
    }
}
```

```

// INSTRUCCIÓN 1
invertir_arr_num(arr_num, longitud);

// INSTRUCCIÓN 2
invertir_arr_punt(arr_punt, longitud);

return 0;
}

```

- a) Analice y asegúrese de comprender el funcionamiento del procedimiento `invertir_arr_punt`: ¿qué sucede cuando a un puntero se lo auto-incrementa/decrementa? Un arreglo de elementos, ¿puede ser manipulado con aritmética de punteros?
- b) Reescriba el procedimiento `invertir_arr_punt` de forma tal que respete el siguiente prototipo: `invertir_arr_punt(int * arr [], int size)`.
- c) Bosqueje esquemáticamente cómo quedan compuestas las celdas de memoria de los arreglos `arr_num` y `arr_punt` para los incisos a continuación. Para esto, considere que los arreglos `arr_num` y `arr_punt` ocupan direcciones de memoria consecutivas a partir de la dirección 0.
 - 1) Cuando de la función `main` se ejecuta únicamente el bloque de instrucciones descrito como *inicialización*.
 - 2) Cuando de la función `main` se ejecutan los bloques de instrucciones descritos como *inicialización* e *instrucción 1*.
 - 3) Cuando de la función `main` se ejecutan los bloques de instrucciones descritos como *inicialización* e *instrucción 2*.
 - 4) Cuando de la función `main` se ejecutan los bloques de instrucciones descritos como *inicialización*, *instrucción 1* e *instrucción 2*.

8. Sabiendo que la función `fopen` se define de la siguiente manera:

```

■ FILE *fopen(const char *filename, const char *mode);

```

Donde `fopen` abre el archivo especificado por el nombre '`filename`' para que sea manipulado en modo '`mode`'. Indicar el significado de la función para los modos: "`r`", "`w`", "`a`", "`r+`", "`w+`" y "`a+`".

9. Implementar un programa que reciba por la línea de comandos el nombre de dos archivos (f_1 y f_2), muestre por pantalla el contenido del archivo f_1 , copie el contenido de f_1 en el archivo f_2 y muestre posteriormente el contenido de este último.
10. Implementar un programa que dado un archivo de texto y una palabra en la línea de comando muestre por pantalla la primer línea que contenga la palabra buscada.
11. Escribir un programa que al recibir el nombre de un archivo de texto en la línea de comandos le agregue el número de línea actual a cada una de las líneas que componen al archivo.

Referencias

- [KR88] KERNIGHAN, B., AND RITCHIE, D. *The C Programming Language*, second ed. Prentice-Hall, 1988.

[Wik] *C Programming*. https://en.wikibooks.org/wiki/C_Programming
Pdf: https://upload.wikimedia.org/wikipedia/commons/0/07/C_Programming.pdf.